

Documentação site www.duvid.com.br

Sumário

Resumo site duvid.....	4
1. Arquivos do Núcleo (Core).....	5
📁 2. Arquivos do Blog (Estrutura de Conteúdo).....	5
📖 3. Mapa de Ativos (Assets).....	6
👤 Destaque Técnico: O Fluxo de Dados.....	6
Sistema Duvid Geografia: Guia de Arquitetura.....	7
🏗️ 1. Core & Banco de Dados (Os Motores).....	7
🎨 2. Interface & Áudio (Os Sentidos).....	7
📖 3. Lógica de Conteúdo (As Engrenagens).....	8
🔧 4. Fluxo de Trabalho Sênior (Clean Code).....	8
💡 Status Atual do Projeto.....	8
5. Módulo do Blog (Atualidade e Conteúdo).....	9
Javascript.....	9
jstextos-padrao.sj.....	9
1. Gestão de Identidade e Sessão.....	9
2. Motor de Gamificação e Feedback.....	10
3. Controle de Navegação e Progresso Interno.....	10
4. Finalização e "Congelamento" da Aula.....	10
Observação Técnica para sua Documentação:.....	11
aulas-geral.js.....	11
1. Carregamento e Renderização de Dados (JSON).....	11
2. Painel de Experiência do Usuário (Boas-vindas).....	11
3. Sistema de Conquistas e Gamificação Global.....	12

4. Utilitários de Interface (UI).....	12
Destaque Técnico para a Documentação:.....	12
carregar.js.....	13
1. Sistema de Injeção de Componentes Globais.....	13
2. Ferramentas de Acessibilidade e Preferências.....	13
3. Navegação e Utilidades de Interface (UX).....	13
4. Automação de Metadados de Rodapé.....	14
Destaque Técnico para a Documentação:.....	14
1. HUB de Identificação e Boas-Vindas.....	14
2. Sistema de Resumo Geral e Troféus (resumo-geral).....	14
3. Dinâmica de Inspiração (Motor de Frases).....	15
4. Integração de Componentes Globais.....	15
Jsquestoes-padrao.js.....	16
Resumo:.....	16
Por que deletar é melhor que deixar em branco?.....	17
O que acontece se eu esquecer e deixar o campo?.....	17
Acessando o PHP:.....	18
duvid-core.js.....	19
Modelo json questões:.....	21
🔧 Comando para o VS Code.....	24
🤔 O que essa lógica faz?.....	24
PRÓXIMAS IMPLEMENTAÇÕES.....	25
Mapa de Implementação Duvid 2026.....	25
Camada 1: O Ciclo de Retenção (O que faz o aluno voltar).....	25
Camada 2: O Ciclo de Engajamento (O que faz o aluno querer ganhar).....	25
Camada 3: Inteligência e UX (O que facilita a vida do aluno).....	25
💡 Novas Sugestões de Mestre.....	25
1. Sistema de "Combo" de Acertos.....	25

2. O "Dicionário" (Glossário Interativo).....	26
3. Modo "Speedrun" (Desafio de Tempo).....	26
4. Personalização de Avatar (O Globinho).....	26
5. Explorar Duvid – inserir um card Atividades.....	26
6. Inserir mais dados via API.....	26
7. Pesquisar sobre segurança de API Keys e Variáveis de Ambiente".....	26
Prompts.....	29
ícones.....	29
Infográfico.....	29
Prompt para as imagens das questões:.....	30

Resumo site duvid

O **Duvid Geografia** é um site gratuito e inovador, feito sob medida para a sala de aula. A plataforma oferece mais de **1000 questões atualizadas** e dezenas de **textos autorais interativos** com correção automática, comentários e sugestões de atividades pedagógicas.

O projeto utiliza elementos de **Gamificação (RPG)** para transformar o aprendizado em uma jornada, com sistema de progresso dinâmico, recompensas (**Globinhos**) e níveis de patente. Além do site, o ecossistema conta com aplicativos para Android que funcionam **100% offline**, garantindo o acesso ao conhecimento em qualquer lugar.

O **Duvid Geografia** é uma Progressive Web Application (PWA) desenvolvida com foco em **baixa latência e acessibilidade**. A arquitetura é baseada em um núcleo de **JavaScript Vanilla** e **W3.CSS**, garantindo performance máxima mesmo em dispositivos com hardware limitado.

Destaques do Motor:

- **Persistência de Dados:** Utiliza o sistema DuvidDB baseado em **LocalStorage**, permitindo que o progresso do aluno (XP, Níveis e Conclusões) seja mantido sem a necessidade de um servidor de banco de dados constante.
- **Arquitetura Gamificada:** Motor de níveis baseado em cálculos dinâmicos de saldo total, com atualização de patentes em tempo real via DOM.
- **Integração de APIs:** Dashboard de monitoramento global integrado via **Fetch API** (OpenWeather, AwesomeAPI) c
- **Mobilidade:** Desenvolvido para funcionar em ecossistema híbrido, com espelhamento total das funcionalidades nos aplicativos Android para uso **100% offline**.

1. Arquivos do Núcleo (Core)

Estes são os arquivos que fazem o "motor" do site girar na Home e no Painel do Usuário.

Arquivo	Função Técnica	Estado Atual
home.html	Estrutura principal da Home, Dashboard de 8/4 colunas e Seção de Monitoramento Global.	Atualizado
duvid-db.js	O "Cérebro". Contém o objeto DuvidDB que gerencia o LocalStorage, XP, Níveis e as Patentes.	Clean Code
duvid-ui.js	A "Interface". Controla os widgets de Clima, População, Dólar e a geração visual dos troféus de Pixel Art.	Gamificado
duvid_api.php	O "Escudo". Proxy em PHP que esconde sua API Key da OpenWeather e faz a ponte segura com o servidor.	Blindado (ainda não fiz)
index-estilo.css	A "Estética". Contém as regras de image-rendering para Pixel Art e os efeitos de hover nos cards.	Pixel-Perfect

2. Arquivos do Blog (Estrutura de Conteúdo)

O Blog não é apenas um repositório de textos; no Duvid, ele é uma **fonte de XP**.

Arquivo	Função Técnica	O que faremos amanhã
blog/blog.html	A vitrine. Lista os artigos disponíveis em cards (estilo o "Explorar").	Adicionar selo de "Lido" dinâmico.
blog/artigos/	Pasta que contém os arquivos HTML de cada texto autoral (ex: geopolitica.html).	Integrar o botão "Concluir Leitura".
blog-engine.js	Lógica que verifica se o aluno já leu o texto e libera os "Globinhos" de recompensa.	Criar a função de gatilho de XP. (ainda não implementei)

3. Mapa de Ativos (Assets)

Onde moram as imagens que dão a cara de RPG ao projeto.

- **fotoIndex/icones/**: Pasta sagrada dos ícones. Aqui estão as patentes (semente, lenda, etc.) e os ícones dos widgets (clima, dólar).
- **fotoIndex/imagensCapa/**: Capas dos anos letivos (1º, 2º e 3º ano).
- **fotoIndex/autores/**: Fotos dos geógrafos para o painel de citações dinâmicas.

Destaque Técnico: O Fluxo de Dados

Para o resumo ficar completo, entenda o caminho que o dado faz:

1. **Input**: O aluno termina uma tarefa ou lê um blog.
2. **Processamento**: O duvid-ui.js chama o DuvidDB.salvarConclusao().
3. **Persistência**: O dado é gravado no localStorage do navegador (Offline-first).
4. **Feedback**: O gerarHtmlTrofeus() relê o saldo e atualiza os ícones de cinza para colorido instantaneamente.

Sistema Duvid Geografia: Guia de Arquitetura

O site **Duvid** é uma plataforma de aprendizagem de Geografia gamificada (RPG) baseada em progresso dinâmico, recompensas (Globinhos) e níveis de patente.



1. Core & Banco de Dados (Os Motores)

- **duvid-db.js (O Banco de Dados/Cérebro):**
 - **Responsabilidade:** Único responsável por ler/escrever no localStorage.
 - **Funções Chave:** getGlobinhos(), addGlobinhos(pts), salvarConclusao(id, tipo), getProgressorPG().
 - **Lógica de RPG:** Contém o RANKING_SISTEMA (Patentes de Novato a Lenda) e calcula a XP necessária para subir de nível.
- **duvid-core.js (O Orquestrador):**
 - **Responsabilidade:** Gerencia o carregamento inicial da página (DOMContentLoaded).
 - **Funções Chave:** verificarStatusAula(id), gerenciarIdentificacaoHome(), executarReset().
 - **Fluxo:** Identifica se o aluno está na Home ou em uma Aula e dispara as funções de interface necessárias.



2. Interface & Áudio (Os Sentidos)

- **duvid-ui.js (O Maestro Visual):**
 - **Responsabilidade:** Toda e qualquer manipulação de DOM, animações e efeitos visuais.
 - **Funções Chave:** * executarGatilhoResultado(correto, pts): Centraliza som + confete + animação + pontos.
 - desenharTrilhaNiveis(): Desenha a trilha estilo Candy Crush com Auto-Scroll.
 - feedbackVisualAcerto() / feedbackVisualErro(): Gira o globinho ou treme o saldo em vermelho.
 - dispararComemoracao(): Motor de 10 efeitos aleatórios de confete.
- **duvid-audio.js (A Vitrola):**

- **Responsabilidade:** Gerenciar todos os efeitos sonoros.
- **Funções Chave:** `playSom('acerto'|'erro'|'click')`, `playSomFinal(aprovado)`.

3. Lógica de Conteúdo (As Engrenagens)

- **jstextos-padrao.js (Motor das Aulas):**
 - **Responsabilidade:** Controla o progresso de leitura, questões internas e o "Modo Cinza" ao concluir a aula.
 - **Funções Chave:** `validarRadio()`, `ProcessarResposta()`, `mostraCinza()`, `injetarMetadadosAula()`.
 - **Destaque:** Agora utiliza o `executarGatilhoResultado` para simplificar a comunicação com o UI.
- **jsquestoes.js (Simulador de Questões):**
 - **Responsabilidade:** Carrega JSONs externos de questões e gerencia o fluxo de simulados.
 - **Funções Chave:** `carregarDados(id)`, `renderizarQuestao()`, `verificar()`, `finalizar()`.

4. Fluxo de Trabalho Sênior (Clean Code)

1. **Separação de Preocupações:** Funções de lógica (DB) nunca devem mexer em cores; funções de UI nunca devem calcular pontos sozinhas.
2. **Padrão de Nomenclatura:** Funções iniciadas com `_` (ex: `_focarNivelAtual`) são utilitários internos do objeto.
3. **Encapsulamento:** Sempre que um bloco de código se repetir (como tremer um botão), ele deve virar uma função no `duvid-ui.js`.
4. **Segurança:** Sempre verificar `if (typeof ... === "function")` para evitar que a falta de um arquivo quebre o site inteiro.

Status Atual do Projeto

-  **Sistema de Pontos:** Funcional e persistente.
-  **Gamificação:** Trilha de níveis dinâmica com frases motivacionais por patente.
-  **Feedback:** Confete, giro de globinho e sons integrados.
-  **Mobile:** Layout de trilha adaptativo com scroll horizontal.

5. Módulo do Blog (Atualidade e Conteúdo)

O Blog funciona de forma assíncrona, carregando os textos e imagens sem precisar criar dezenas de arquivos HTML manuais.

- **artigos.json:** O "banco de dados" do blog. Contém títulos, resumos, datas, caminhos das fotos e o conteúdo dos artigos.
- **galeria.json:** Gerencia as imagens e legendas que aparecem nas seções de fotos ou carrosséis.
- **scripts-blog.js (O Motor do Blog):**
 - **Responsabilidade:** Lê os arquivos JSON e injeta os cards de notícias na `blog.html`.
 - **Padrão Sênior:** Utiliza `fetch()` para carregar os dados e `Template Literals` para montar o HTML dinamicamente.
- **blog-estilo.css:** CSS exclusivo para a área de notícias, garantindo que a leitura seja confortável (tipografia) e que as imagens se adaptem a qualquer tela.
- **Pasta /artigos:** Armazena os arquivos HTML individuais dos artigos completos.
- **Pasta /fotos:** Centraliza as imagens otimizadas para o blog (evitando bagunçar a pasta de fotos do RPG).

Javascript

Arquivo

`jstextos-padrao.sj`

1. Gestão de Identidade e Sessão

Este bloco cuida de quem é o aluno e como o site o reconhece entre as páginas.

- **NomeAlunos:** Captura o nome, salva no `localStorage` (persistência) e atualiza a interface. Se estiver na Home, chama a saudação; se estiver na aula, libera o conteúdo.
- **exibirSaudacao & resetarNome:** Gerenciam a troca de estado na Home (de "Digite seu nome" para "Bem-vindo de volta") e permitem a troca de usuário.

- **injetarMetadadosAula:** Uma função inteligente que lê a URL, busca no JSON o título da aula e injeta automaticamente no `<h1>` e no `<title>` da aba do navegador.

2. Motor de Gamificação e Feedback

Responsável por transformar a leitura em uma atividade interativa com pontuação.

- **ProcessarResposta:** O núcleo das questões. Calcula a nota (soma se acertar, penaliza levemente se errar), toca áudios e gerencia o feedback visual (Globinho triste/feliz).
- **validarRadio:** Atua como um "segurança" das questões, garantindo que o aluno escolheu uma opção antes de processar a resposta.
- **feedbackVisualAcerto:** Cria animações na barra de navegação (o globinho balança e a nota aumenta com zoom) para celebrar o acerto do aluno.
- **Play:** Sistema centralizado de áudio que busca os sons na pasta raiz `/audios/`.

3. Controle de Navegação e Progresso Interno

Controla como o conteúdo é revelado para garantir que o aluno siga a sequência pedagógica.

- **MostrarProximo:** Resolve o problema de containers escondidos. Quando o aluno clica em "Confirmar", ele revela o próximo tópico com um efeito de rolagem suave (*smooth scroll*).
- **addProgressBar:** Calcula dinamicamente a porcentagem de leitura da aula com base na quantidade de elementos com a classe `.topico`.

4. Finalização e "Congelamento" da Aula

Gerencia o que acontece quando o aluno chega ao fim da atividade.

- **mostraCinza:** A função de encerramento. Ela verifica se é a primeira vez que o aluno termina (para não salvar a nota repetidas vezes) e dispara o processo de finalização.
- **desativarBotoes, desativarTextos & desativarImagens:** Criam o efeito visual de "Missão Cumprida". Bloqueiam cliques, deixam o texto cinza e as imagens em preto e branco, focando a atenção do aluno no resultado final.

- **mostrarNota:** Abre o modal final (`id01`) com a mensagem personalizada baseada no desempenho (acima ou abaixo de 6.0).

Observação Técnica para sua Documentação:

Ponte de Dados: O script utiliza o `localStorage` como um banco de dados local, permitindo que a Home saiba quais aulas foram concluídas através de chaves como `concluido_3ano/Textos3/Texto01/tt1.html`.

Arquivo –

`aulas-geral.js`

1. Carregamento e Renderização de Dados (JSON)

Este bloco é responsável por ler os arquivos externos e montar a interface visual de forma automatizada.

- **carregarAulas(ano):** É a função principal. Ela faz o `fetch` (busca) do arquivo JSON correspondente ao ano, processa os dados e gera o HTML de todos os cards de aula.
- **Mapeamento de Estado:** Dentro do `map`, o script verifica no `localStorage` se cada aula individual já foi concluída. Se sim, ele aplica estilos CSS específicos (borda verde, ícone de check e filtro colorido na imagem).
- **Modais Dinâmicos:** Para cada card gerado, o script cria um modal (janela flutuante) exclusivo que contém o resumo do conteúdo e os botões de ação ("Texto/Revisar" e "Questões").

2. Painel de Experiência do Usuário (Boas-vindas)

Gerencia o primeiro contato do aluno com a página de estudos.

- **mostrarProgressoGlobal(aulas, ano):** Constrói o cabeçalho personalizado. Ele resgata o nome do aluno e calcula a porcentagem de conclusão **especificamente para o ano selecionado**.

- **Barra de Progresso Dinâmica:** Gera uma barra visual que usa a propriedade `transition: width 1s` para criar um efeito de preenchimento suave ao carregar.

3. Sistema de Conquistas e Gamificação Global

Controla o rastreamento de longo prazo do aluno em todo o site.

- **atualizarResumoHome():** Esta função é o "quadro de medalhas". Ela percorre o progresso dos três anos simultaneamente (1º, 2º e 3º) e atualiza as barras de resumo na página inicial.
- **Lógica de Troféus:** Monitora se o total de aulas concluídas atingiu o total disponível. Se o aluno completar um ano inteiro, o script dispara um som de vitória (`notaFinal.mp3`) e exibe um ícone de conquista com animação de zoom.

4. Utilitários de Interface (UI)

Funções de suporte para a interatividade da página.

- **ExpandeDiv:** Gerencia a abertura e o fechamento dos modais de aula, garantindo que, ao abrir uma aula, todas as outras janelas abertas sejam fechadas automaticamente para manter a tela limpa.
- **window.onclick:** Um "ouvinte" global que detecta cliques fora da área do modal para fechá-lo, melhorando a usabilidade (UX).

Destaque Técnico para a Documentação:

Arquitetura Baseada em Dados: Ao contrário do modelo antigo onde cada card era escrito manualmente no HTML, este motor permite que o Professor Leandro adicione ou remova aulas apenas editando um arquivo JSON, sem precisar mexer no código JavaScript ou HTML.

carregar.js

1. Sistema de Injeção de Componentes Globais

Este é o "coração" da manutenção simplificada do site.

- **injetarComponentesGlobais:** Esta função utiliza o método `fetch` para buscar arquivos HTML externos (`header.html` e `footer.html`) e injetá-los nos *placeholders* (espaços reservados) de cada página.
- **Gestão de Estado do Header:** O script é inteligente o suficiente para saber se o aluno está em uma aula ou na Home. Através da função `verificarSeEhAula()`, ele decide se deve mostrar ou esconder o **Painel de Pontos** (o globinho e a nota) na barra de navegação.
- **Sincronização de Dados:** Garante que o contador de pontos (`notaFixa`) exiba o valor real vindo do motor de questões assim que o header é carregado.

2. Ferramentas de Acessibilidade e Preferências

Responsável por adaptar o site às necessidades e gostos do usuário, salvando as escolhas no navegador.

- **Controle de Fonte (`inicializarControleFonte`):** Permite que o aluno aumente ou diminua o tamanho do texto (entre 0.8rem e 1.5rem). O tamanho escolhido é salvo no `localStorage`, garantindo que o aluno não precise ajustar a fonte toda vez que mudar de página.
- **Modo Escuro (Dark Mode):** Gerencia a troca de temas (Claro/Escuro). Ele altera as classes do `body` e troca o ícone da barra (Lua/Sol). Assim como a fonte, o tema escolhido é persistente (salvo no navegador).

3. Navegação e Utilidades de Interface (UX)

Melhora a experiência de leitura em textos longos.

- **Botão "Voltar ao Topo":** O script monitora a rolagem da página através do `window.onscroll`. Quando o usuário desce mais de 400px, um botão flutuante aparece.
- **`voltarAoTopo`:** Executa uma rolagem suave (*smooth scroll*) de volta ao início da página, evitando saltos bruscos que podem desorientar a leitura.

4. Automação de Metadados de Rodapé

- **Atualização de Data:** O script localiza o ID `ano-atual` no rodapé e insere automaticamente o ano vigente usando `new Date().getFullYear()`, evitando que o site pareça desatualizado.

Destaque Técnico para a Documentação:

Modularidade e Performance: Ao usar o carregamento assíncrono (`async/await`) para os componentes globais, o script evita que o carregamento do Header ou Footer trave a leitura do conteúdo principal da aula. Além disso, o uso de `defer` ou `DOMContentLoaded` garante que os scripts só mexam no HTML quando ele estiver pronto.

home.html

1. HUB de Identificação e Boas-Vindas

Diferente das páginas de aula, a Home gerencia o estado global do aluno no site.

- **Gestão de Identidade (container - login):** Atua como a porta de entrada. Se o aluno é novo, exibe o campo de captura de nome. Se é um aluno recorrente, o script `jestextos-padrao.js` (carregado no `head`) intercepta o carregamento e substitui o formulário por uma saudação personalizada ("Bem-vindo de volta, Leandro!").
- **Persistência Global:** Como o `localStorage` é compartilhado por todo o domínio, o nome digitado nesta seção da Home passa a ser a variável `nomeEstudante` disponível em todas as subpáginas e exercícios.

2. Sistema de Resumo Geral e Troféus (resumo-geral)

Esta é a seção exclusiva da Home que oferece uma visão panorâmica de todo o Ensino Médio.

- **Monitoramento Multi-Série:** Enquanto as páginas de ano mostram apenas o progresso daquele ano, a Home executa o motor `atualizarResumoHome()`, que lê as conclusões dos três anos simultaneamente.
- **Visualização de Metas:** Utiliza barras de progresso coloridas (Verde para 1º, Azul para 2º e Laranja para 3º) para motivar o aluno.

- **Gatilho de Conquistas:** É aqui que o script verifica se a contagem de aulas concluídas atingiu o máximo do JSON. Caso positivo, revela o ícone de troféu (fa-trophy) com a animação w3-animate-zoom e dispara o áudio de celebração.

3. Dinâmica de Inspiração (Motor de Frases)

Um componente focado na experiência do usuário (UX) e na ambientação pedagógica.

- **Renderização Assíncrona:** A função `carregarFrase()` utiliza `fetch` para ler um banco de dados externo (`frases.json`). Isso permite que o Professor Leandro atualize ou adicione novas citações geográficas sem precisar mexer no código da Home.
- **Ciclo de Exposição:** Implementa um `setInterval` de 10 segundos com transições de opacidade (`opacity`), garantindo que o aluno receba diferentes estímulos intelectuais (frases de Milton Santos, Harvey, etc.) enquanto navega.

4. Integração de Componentes Globais

- **Injeção de Cabeçalho e Rodapé:** Através do `carregar.js`, a Home recebe dinamicamente o menu de navegação e o rodapé.
- **Segurança de Contexto:** A função `verificarSeEhAula()` dentro do componente global identifica que a Home **não** é uma lição interativa e, por isso, oculta automaticamente o globinho de pontos da Navbar, mantendo a interface limpa.

Jsquestoes-padrao.js

O Arquivo json deve estar na pasta da raiz questoes. Dentro dele tem a pasta 1ano e seus respectivos json 101, 102 103

Resumo:

- **Nome do Arquivo:** 101.json, 102.json (Este é o ID da Aula).
- **Dentro do JSON:** Pode usar `id: 1` até `id: 10` em todos os arquivos.
- **Fotos:** Use sempre o prefixo da aula (`102_q1.png`).

Arquivos questões

```
[
  {
    "id": 101,
    "ano": "ENEM 2024",
    "imagem_apoio": "1ano/img/101_q1.png",
    "pergunta": "Pergunta com imagem e ajuda.",
    "alternativas": ["a", "b", "c"],
    "correta": 0,
    "comentario": "...",
    "ajuda": { "texto": "Dica!", "imagem": "1ano/img/dica101.png" }
  },
  {
    "id": 102,
    "ano": "FUVEST",
    "pergunta": "Questão pura (Sem imagem e sem ajuda).",
    "alternativas": ["Sim", "Não"],
    "correta": 1,
    "comentario": "..."
  }
]
```



```
// Note que aqui NÃO TEM imagem_apoio nem ajuda. O script vai ignorar.
},
{
  "id": 103,
  "ano": "UnB",
  "pergunta": "Questão só com ajuda de texto (sem foto na ajuda).",
  "alternativas": ["Certo", "Errado"],
  "correta": 0,
  "comentario": "...",
  "ajuda": { "texto": "Lembre-se da técnica X." }
  // Aqui não tem imagem_apoio nem imagem dentro da ajuda.
}
]
```

```
</p>\n\n\t\t\t<p>
```

Espaço para o json das questoes

Por que deletar é melhor que deixar em branco?

Se você deixar `"imagem_apoio": ""`, o script pode tentar carregar uma imagem vazia e mostrar um ícone de erro. Deletando a linha ou deixando o campo como `null` ou `undefined`, o nosso código faz o bloco desaparecer completamente, mantendo o layout perfeito.

O que acontece se eu esquecer e deixar o campo?

Não se preocupe! Colocamos o `onerror="this.style.display='none'"` nas tags de imagem. Se o caminho estiver errado ou vazio, a imagem simplesmente se "auto-esconde" e o aluno não percebe o erro.

Acessando o PHP:

Passo a Passo para rodar o site PHP localmente com XAMPP

1. Instalar o XAMPP

Se ainda não instalou:

- Acesse <https://www.apachefriends.org>
- Baixe e instale o XAMPP para seu sistema.

2. Iniciar os serviços

Abra o **Painel de Controle do XAMPP** e **inicie os módulos**:

-  Apache (servidor web)
-  MySQL (se você usa banco de dados)

3. Salvar seu projeto na pasta correta

- Coloque seus arquivos PHP dentro da pasta:

`http://localhost/duvid/`

duvid-core.js

```

/* =====
DUVID CORE - Central de Gamificação, Sons e LocalStorage
===== */

// 1. CONFIGURAÇÕES DE SOM
const SONS_VITORIA = ['/audios/acerto1.mp3', '/audios/acerto2.mp3'];
const SONS_ERRO = ['/audios/erro1.mp3', '/audios/erro2.mp3'];

function playSom(tipo) {
  const lista = (tipo === 'acerto') ? SONS_VITORIA : SONS_ERRO;
  const sorteado = lista[Math.floor(Math.random() * lista.length)];
  const audio = new Audio(sorteado);
  audio.volume = (tipo === 'acerto') ? 0.7 : 0.4;
  audio.play().catch(e => console.log("Áudio pendente de interação."));
}

// 2. CENTRAL DE COMEMORAÇÕES (Os 10 Efeitos de Confete)
function dispararConfete() {
  const estilo = Math.floor(Math.random() * 10) + 1;
  // ... aqui entra aquela função switch(estilo) com os 10 efeitos que criamos ...
  console.log("Efeito de confete tipo: " + estilo);
}

// 3. CENTRAL DE ECONOMIA (LocalStorage)
const DB_CHAVE = "duvid_globinhos";

```

```

const DuvidDB = {
  getGlobinhos: function() {
    return parseInt(localStorage.getItem(DB_CHAVE)) || 0;
  },

  addGlobinhos: function(qtd) {
    let atual = this.getGlobinhos();
    localStorage.setItem(DB_CHAVE, atual + qtd);
    this.verificarConquistas(); // Checa se ganhou selos/banners
  },

  salvarConclusao: function(idAula, tipo) {
    // tipo pode ser 'texto' ou 'questoes'
    localStorage.setItem(`concluido_${tipo}_${idAula}`, "true");
  },

  verificarConquistas: function() {
    let saldo = this.getGlobinhos();
    // Lógica dos Banners de Conquista
    if (saldo >= 5000) localStorage.setItem("patente", "Geógrafo Sênior");
    // ... etc
  }
};

```

Modelo json questões:

```
{
  "id": 0,
  "instituicao": "UERJ",
  "ano": "(Ano)",
  "dificuldade": "Difícil",
  "tags": ["Categorias Geográficas", "Milton Santos", "Espaço Geográfico"],
  "texto_apoio": "<p>Insira aqui o texto base com aspas escapadas ou tags HTML simples.</p>",
  "fonte_apoio": "AUTOR, Nome. Título do Livro. Cidade: Editora, Ano, Página.",
  "imagem_apoio": "caminho/da/imagem1.png",
  "legenda_imagem": "Figura 1: Descrição técnica da primeira imagem.",
  "imagem_apoio_2": "caminho/da/imagem2.png",
  "legenda_imagem_2": "Figura 2: Descrição técnica da segunda imagem.",
  "pergunta": "Enunciado da questão aqui.",
  "alternativas": [
    "Alternativa A",
    "Alternativa B",
    "Alternativa C",
    "Alternativa D",
    "Alternativa E"
  ],
  "correta": 0,
  "ajuda": "Dica curta que estimula o raciocínio sem dar a resposta.",
}
```

```
"comentario": "<p>Explicação detalhada de cada  
alternativa...</p>",
```

```
"imagem_comentario": "caminho/da/imagem_feedback.png",
```

```
"legenda_comentario": "Mapa mental ou esquema resumindo o  
conteúdo da questão."
```

```
}
```

INSERIR NO TOP DOS TEXTOS

Confetti:

```
<script src="https://cdn.jsdelivr.net/npm/canvas-  
confetti@1.9.2/dist/confetti.browser.min.js"></script>
```

Teste

```
"linkQuestoes": "3ano/Aulas3/Aula01/t1.html"
```

PROMPT

"Estou trabalhando no projeto **Duvid**, um sistema de ensino gamificado. Preciso que você assuma o papel de desenvolvedor sênior e considere as seguintes especificações atuais:

1. **Banco de Dados (DuvidDB):** Utilizamos um objeto global DuvidDB que gerencia o `localStorage`. Ele possui os métodos `getGlobinhos()`, `addGlobinhos(n)`, `salvarConclusao(id, tipo)` e `getNome()`.
2. **Identidade Visual:** Usamos W3.CSS com personalizações no `ModeloCss.css`. A Navbar é fixa (`w3-top`) e unificada. O Dark Mode é controlado por uma classe no `body`.
3. **Painel de Pontos (Navbar):** Estrutura de IDs na Navbar:
 - `#saldoTotalHeader`: Dentro de uma `.caixa-total-dourada` (Saldo acumulado do aluno).
 - `#notaFixa`: Pontos ganhos na aula atual (Texto ou Questões).
 - `#painel-pontos`: O container que agrupa ambos.
4. **Páginas de Aula:** Os arquivos de texto e questões recebem um `?id=XXX` via URL. O script usa `aulaID = new URLSearchParams(window.location.search).get('id')` para gravar o progresso.
5. **Função Global de Sincronização:** A função `atualizarGlobinhosGeral()` no `duvid-core.js` deve atualizar simultaneamente o `#saldoTotalHeader` (Navbar) e o `#valor-total-central` (Card da Home).
6. **Fluxo de Finalização:** Nas questões, o `finalizar()` injeta um layout de botões empilhados (Refazer / Home) e garante que a barra de progresso chegue a 100%.

Com base nisso, pode me ajudar com a seguinte tarefa: [INSERIR SUA DÚVIDA AQUI]"

Dica expressão regular:

Se você já tem o arquivo JSON aberto no VS Code e quer remover as letras duplicadas de todas as alternativas de uma vez, a melhor forma é usar o **Localizar e Substituir com Regex** (Expressões Regulares).

Siga este passo a passo:

 Comando para o VS Code

1. Pressione `Ctrl + H` para abrir a janela de **Substituir**.
2. Clique no ícone de asterisco `.` `*` (ou use o atalho `Alt + R`) para ativar o modo **Expressão Regular**.
3. No campo **Localizar**, cole este código: `"([a-e])(\\.|\\-|\\s|\\s)+"`
4. No campo **Substituir**, cole apenas um caractere de aspas: `"`
5. Clique no botão **Substituir Tudo** (o ícone com dois quadrados sobrepostos).

 O que essa lógica faz?

- `"` : Procura a aspa que inicia o texto da alternativa.
- `([a-e])` : Procura qualquer letra de 'a' até 'e'.
- `[\\.|\\-|\\s|\\s]+` : Procura por um parêntese, ponto, traço ou espaço que venha logo depois da letra.
- **Resultado:** Ele encontra `"a)` e substitui por `"`, removendo a letra e mantendo o JSON válido.

PRÓXIMAS IMPLEMENTAÇÕES

Mapa de Implementação Duvid 2026

Camada 1: O Ciclo de Retenção (O que faz o aluno voltar)

- **Notificações de Progresso:** Um pequeno alerta na Home: *"Faltam apenas 20 XP para o nível 'Geógrafo Senior'"*. (já feito)

Camada 2: O Ciclo de Engajamento (O que faz o aluno querer ganhar)

- **Feedback "Juicy":** Animar o contador de globinhos com tremedeira e brilho no acerto. (já feito)
- **Badges de Especialista:** Medalhas visuais por categoria (Climatologia, Demografia, Geopolítica).
- **Recompensa de Leitura:** O botão "Reivindicar Pontos" ao final de textos longos para valorizar o esforço.

Camada 3: Inteligência e UX (O que facilita a vida do aluno)

- **Save Point Automático:** O botão "Continuar de onde parou" usando `localStorage`.
- **Gráfico de Radar (Desempenho):** Um gráfico que mostra os pontos fortes e fracos em tempo real.
- **Busca Dinâmica:** Filtro de palavras-chave nos JSONs.

💡 Novas Sugestões de Mestre

Para deixar o site ainda mais robusto e "viciante", aqui estão 4 novas implementações:

1. Sistema de "Combo" de Acertos

- **Como funciona:** Se o aluno acertar 3 questões seguidas sem errar, os globinhos ganham um multiplicador (x1.5, x2.0).
- **Psicologia:** Cria um estado de "Fluxo". O aluno fica muito mais concentrado para não quebrar a sequência de bônus.

2. O "Dicionário" (Glossário Interativo)

- **Como funciona:** Sempre que um termo técnico aparecer no texto (ex: *Rugosidades, Meio Técnico-Científico-Informacional*), ele é um link que abre um pequeno "Card de Lore" (estilo RPG) explicando o conceito de forma simples.
- **Psicologia:** Facilita a compreensão sem que o aluno precise sair do site para pesquisar, mantendo-o dentro do seu ecossistema.

3. Modo "Speedrun" (Desafio de Tempo)

- **Como funciona:** Um modo especial onde o aluno tem 30 segundos para responder cada questão de um tema. No final, ele ganha globinhos extras baseados na velocidade.
- **Psicologia:** Treina a agilidade para o ENEM e adiciona uma camada de adrenalina ao estudo.

4. Personalização de Avatar (O Globinho)

- **Como funciona:** Na loja de itens, além de temas, o aluno pode comprar "skins" para o seu globinho (ex: Globinho com óculos, Globinho Astronauta, Globinho Explorador).
- **Psicologia: O Efeito de Propriedade.** Quando o aluno personaliza o seu perfil, ele se sente dono do progresso e a chance de ele abandonar o site diminui drasticamente.

5. Explorar Duvid – inserir um card Atividades.

Vai abrir uma página com todas as atividades práticas para os professores imprimirem em pdf., que existem nas aulas.

6. Inserir mais dados via API

– População brasileira, índice IBOVESPA.

7. Pesquisar sobre segurança de API Keys e Variáveis de Ambiente"

SEMANA 3 — ENGAJAMENTO ESTILO DUOLINGO (O QUE PRENDE O ALUNO)

9

XP flutuante ao acertar

" +10 XP" sobe e some sobre o saldo. Feedback imediato é o gatilho de recompensa mais forte do Duolingo. Fácil de implementar.

Duolingo

10**revelarParentese — expandir o uso**

O mecanismo mais original do projeto. Transformar em padrão obrigatório nos textos novos, com animação própria e XP maior.

Duolingo**11****Sistema de vidas com corações**

3 corações por sessão de questões. Zerar leva à tela "tente novamente". Cria urgência e atenção — o aluno para de chutar.

Duolingo**12****Streak diário**

Dias consecutivos de acesso. O Duolingo deve 80% do engajamento a isso. Com PHP/MySQL fica ainda mais confiável que no localStorage.

Duolingo**13****Modal de conclusão mais rico**

Adicionar streak atual, XP ganho na sessão e patente. O momento de fechamento da aula é o mais importante para fixar o hábito.

Duolingo

FASE PHP — DEPOIS DA MIGRAÇÃO ESTAR ESTÁVEL**14****Tabelas MySQL — alunos, progresso, aulas**

Substituir localStorage e JSONs. A decisão de login com senha ou só por nome define toda a estrutura.

PHP**15****API PHP para salvar progresso**

Endpoint simples que o JS chama via `fetch` para salvar globinhos e conclusões. O DuvidDB vira um cliente desta API.

PHP**16****Ranking global de alunos**

Top alunos por globinhos e streak. Impossível sem banco. É o recurso que mais vai criar competição saudável entre alunos.

O QUE MELHORAR:

Mapa de Organização Sugerido

Se o arquivo passar de 500 linhas, recomendo dividir assim:

1. **duvid-db.js**: Apenas o objeto DuvidDB e as constantes de recompensas.
2. **duvid-ui.js**: atualizarInterface, dispararComemoracao e animarContador.
3. **duvid-audio.js**: Todas as funções de playSom.

🔴 **veredito:** O código está funcional, inteligente e pronto para ser escalado. O fato de você ter percebido que ele está "grande" mostra que seu instinto de programador está evoluindo para a organização!

Prompts

ícones

Redesenhe o ícone em anexo em pixel art nítido, de alta resolução (640x430 px) com um fundo contextual. O estilo deve ser 'pixel art polido' (polished pixel art), com cores ricas e sombras definidas (sem borrão) para criar volume e profundidade. O objeto deve ser claramente reconhecível, com detalhes aprimorados em relação a versões simples. Preencha todo o espaço da imagem.

Faça uma imagem em pixel art nítido, de alta resolução (640x430 px) com um fundo contextual. O estilo deve ser 'pixel art polido' (polished pixel art), com cores ricas e sombras definidas (sem borrão) para criar volume e profundidade. O objeto deve ser claramente reconhecível, com detalhes aprimorados em relação a versões simples. Preencha todo o espaço da imagem. Não precisa inserir textos na imagem. Deve aparecer esses temas em uma bela composição:

Infográfico

"Infográfico educativo vertical (proporção 9:16) no estilo 'roadmap' ou caminho sinuoso, inspirado em materiais didáticos modernos. O design deve apresentar uma estrada central em perspectiva que sobe pela imagem.

TEMA PRINCIPAL: [Insira o Título da Aula aqui, ex: Cartografia e Localização].

DETALHES VISUAIS:

1. Cores: Paleta de cores sóbria e profissional (tons de azul, verde água e cinza claro).
2. Elementos: Ícones limpos, pequenos mapas, gráficos simplificados e personagens minimalistas (estilo flat design) interagindo com o conteúdo.
3. Texto: Cabeçalhos claros em destaque para cada número. Incluir pequenas caixas de texto com bullet points explicativos ao lado de cada ícone.

Prompt para as imagens das questões:

Pegue a ideia central da questão acima e faça a imagem com essas características:

Estilo: Ilustração digital educacional limpa, traços nítidos, cores vibrantes mas paleta profissional, estilo cartum pixel art de livro didático moderno. Centralizado, foco no conceito. Composição horizontal otimizada para uma altura de 580 pixels. *paleta profissional do site duvid.com.br*". Uma imagem quadrada, 580 x 580 px, sem texto dentro da imagem. A ideia é uma figura que ilustra o que a questão pede.

